UNIVERSIDAD DE SAN BUENAVENTURA
Tecnología en Desarrollo de Software · VI semestre
Programación Avanzada y Aplicada

Manual técnico

Instalación, compilación, configuración, datos y mantenimiento

Proyecto final: **AsistenteIA**, asistente académico móvil con .NET MAUI y una API REST de IA

Autores: Jeferson Wilderman González Tenjo, María Alejandra Gómez y José Leandro Ocampo
Docente: Mg. Sandra Esperanza Sánchez Barragán
Fecha: 16 de octubre de 2026
Versión: rama `main` (versión final del proyecto)

Introducción

Este manual está dirigido a desarrolladores que necesiten compilar, configurar, mantener o extender AsistenteIA. Complementa el documento de Arquitectura del software.

Estructura de la solución

```
Proyecto final/
├─ AsistenteIA.slnx             Solución de Visual Studio
├─ AsistenteIA/                 Proyecto .NET MAUI (Models, Services, ViewModels, Views, Controls, ...)
├─ AsistenteIA.Tests/           Pruebas unitarias xUnit (dotnet test AsistenteIA.Tests)
├─ scripts/verificar-calidad.ps1  Verificaciones automáticas de calidad y seguridad
├─ docs/                        Requisitos, arquitectura, API, riesgos, pruebas, manuales
│  ├─ evidencias/v2/            Capturas de la versión actual
│  ├─ informes/                 Informes en PDF y sus generadores
│  └─ manuales/                 Manual de usuario en PDF y su generador
└─ README.md
```

Requisitos de desarrollo

- Windows 10/11.
- Visual Studio 2022 17.14+ o 2026 con la carga **Desarrollo de la interfaz de usuario de aplicaciones multiplataforma de .NET**, o .NET SDK 10.0.4xx con `dotnet workload install maui-windows android`.
- Para Android: SDK de Android 36 y un emulador o dispositivo con Android 5.0 (API 21) o superior.

Paquetes NuGet

Paquete	Versión	Uso
Microsoft.Maui.Controls	10.0.20	Framework
CommunityToolkit.Mvvm	8.4.2	MVVM con generadores de código
sqlite-net-pcl	1.11.285	ORM de SQLite
SQLitePCLRaw.bundle_green	2.1.11	Proveedor nativo de SQLite
SQLitePCLRaw.lib.e_sqlite3 (+ .android)	2.1.14	Fijado para corregir GHSA-2m69-gcr7-jv3q

Compilación

```
dotnet build AsistenteIA/AsistenteIA.csproj -f net10.0-windows10.0.19041.0           # Windows
dotnet build AsistenteIA/AsistenteIA.csproj -f net10.0-windows10.0.19041.0 -t:Run    # Ejecutar
dotnet build AsistenteIA/AsistenteIA.csproj -f net10.0-android                        # Android
dotnet publish AsistenteIA/AsistenteIA.csproj -f net10.0-android -c Release -p:AndroidPackageFormat=apk -o publish/
android   # APK
dotnet publish AsistenteIA/AsistenteIA.csproj -f net10.0-windows10.0.19041.0 -c Release -p:
RuntimeIdentifierOverride=win-x64 -p:SelfContained=true -p:WindowsAppSDKSelfContained=true -o publish/windows   #
Ejecutable de Windows
```

El APK queda en `publish/android/co.edu.usb.asistenteia-Signed.apk` (firmado con la clave de depuración) y el ejecutable de Windows en `publish/windows/AsistenteIA.exe` (`WindowsPackageType=None`, sin empaquetar y autocontenido: no requiere instalar .NET ni Windows App SDK). En Windows se usa `RuntimeIdentifierOverride` y no `-r`, porque `-r` también se aplica a la restauración de Android y falla.

Paquete nativo de SQLite. `SQLitePCLRaw.lib.e_sqlite3` (2.1.14, parcheada) se referencia solo fuera de Android. En Android se usa `SQLitePCLRaw.lib.e_sqlite3.android`: si se incluye el paquete genérico, sus binarios `linux-*` reemplazan en el APK a los de Android y la app se cierra al iniciar (fallo F20).

Configuración

Elemento	Ubicación	Cómo cambiarlo
Proveedores (URL base, modelo por defecto, modelos de respaldo, prefijo de clave)	`Models/AiProvider.cs` (`Pollinations`, `Gemini`)	Código. `AiService` arma la URL completa con la URL base del proveedor activo.
Proveedor activo	Preferences `ProviderId` (`SettingsService.Provider`)	Desde la app (Configuración → Proveedor de IA).
Modelo	Preferences `SelectedModel_{proveedor}`	Desde la app; se guarda uno por proveedor.
Tiempo de espera	Preferences `TimeoutSeconds` (60 s por defecto; se recomienda 120 s con el plan gratuito de Gemini)	Desde la app (de 1 a 120).
Clave	SecureStorage `{proveedor}_api_key` (`gemini_api_key`, `pollinations_api_key`)	Solo desde la app.
Base de datos	`FileSystem.AppDataDirectory/asistenteia.db3`	—
Permisos de Android	`Platforms/Android/AndroidManifest.xml`	`INTERNET`, `ACCESS_NETWORK_STATE`

Ubicación de la base de datos en Windows (app sin empaquetar): `%LOCALAPPDATA%\User Name\co.edu.usb.asistenteia\Data\asistenteia.db3` o la ruta que devuelve `FileSystem.AppDataDirectory`. Se puede abrir con DB Browser for SQLite para comprobar los registros (evidencia de P07).

Esquema SQLite

```
CREATE TABLE ConversationRecord (
  Id INTEGER PRIMARY KEY AUTOINCREMENT,
  Prompt TEXT, Answer TEXT, CreatedAt BIGINT,
  Category TEXT, IsVerified INTEGER, Model TEXT
);
CREATE INDEX ConversationRecord_CreatedAt ON ConversationRecord(CreatedAt);
CREATE INDEX ConversationRecord_Category ON ConversationRecord(Category);

CREATE TABLE PromptTemplate (
  Id INTEGER PRIMARY KEY AUTOINCREMENT,
  Name TEXT, Category TEXT, Text TEXT
);
```

Las tablas y las seis plantillas iniciales se crean en el primer acceso (`DatabaseService.GetConnectionAsync`).

Cambiar de proveedor de IA

Como el formato es compatible con OpenAI Chat Completions:

1. Agregue un `AiProvider` en `Models/AiProvider.cs` con su `Id`, `Name`, `BaseUrl` (terminada en `/`), `DefaultModel`, `KeysUrl`, `FallbackModels` y, si aplica, `KeyPrefix`.
2. Inclúyalo en la lista `AiProvider.All`. Aparecerá en el selector de Configuración con su propia ranura de clave.
3. Si el proveedor filtra modelos o usa otro esquema de autenticación, ajuste `AiService.GetModelsAsync` o el encabezado en `AiService.SendChatAsync`.

Registro de errores (Windows)

`Platforms/Windows/App.xaml.cs` escribe las excepciones no controladas en `errores.log` (carpeta `FileSystem.AppDataDirectory`; en Windows sin empaquetar: `%LOCALAPPDATA%\\User Name\\co.edu.usb.asistenteia\\Data`). Solo guarda la traza del error, nunca la clave ni el contenido de las consultas. Con este registro se encontró el fallo F18.

Pruebas unitarias

El proyecto `AsistenteIA.Tests` (xUnit, .NET 10) prueba las reglas de `AiService` que no dependen de MAUI: validación de la consulta, limpieza del Markdown, mensajes de error HTTP y filtro de modelos (68 pruebas, casos U01–U04 de 05-plan-pruebas.md). Un proyecto de pruebas no puede referenciar una app MAUI, así que esas reglas están en `Services/AiService.Rules.cs` (clase parcial) y el proyecto de pruebas compila ese archivo directamente.

```
dotnet test AsistenteIA.Tests
```

Si se agrega una regla nueva a `AiService` que no use MAUI, conviene ponerla en `AiService.Rules.cs` y cubrirla con una prueba.

Verificación de calidad antes de entregar

`scripts/verificar-calidad.ps1` repite los casos automáticos Q01, Q02, U01–U04 y S01–S05 y termina con código 1 si alguno falla (`-OmitirAndroid` salta Q02):

```
powershell -ExecutionPolicy Bypass -File scripts/verificar-calidad.ps1
```

Pruebas automatizadas de interfaz

Cada control relevante tiene un `AutomationId` (por ejemplo `ChatPromptEditor`, `ChatAskButton`, `SettingsKeyEntry`). Con esos identificadores se puede manejar la app con UI Automation de Windows (o Appium / WinAppDriver) para repetir las pruebas P01–P10.

Seguridad: lista de verificación antes de entregar

- `git grep -nE "sk_[A-Za-z0-9]{10,}|pk_[A-Za-z0-9]{10,}"` no devuelve claves reales.
- Sin carpetas `bin/` ni `obj/` en el repositorio (`.gitignore`).
- Ninguna captura muestra la clave.
- `dotnet list package --vulnerable --include-transitive` sin resultados.

Inyección de dependencias

Para agregar un servicio, ViewModel o página, regístrelo en MauiProgram.cs y recíbalo por el constructor:

```
// HttpClient reutilizable. La URL depende del proveedor elegido (AiProvider) y el tiempo
// de espera por solicitud lo controla AiService; este valor es solo un límite de seguridad.
builder.Services.AddSingleton(new HttpClient
{
    Timeout = TimeSpan.FromSeconds(150)
});

// APIs de la plataforma
builder.Services.AddSingleton(Connectivity.Current);
builder.Services.AddSingleton(Preferences.Default);
builder.Services.AddSingleton(Clipboard.Default);
builder.Services.AddSingleton(Share.Default);

// Servicios
builder.Services.AddSingleton<ApiKeyService>();
builder.Services.AddSingleton<SettingsService>();
builder.Services.AddSingleton<AiService>();
```

```
        builder.Services.AddSingleton<DatabaseService>();
        builder.Services.AddSingleton<IDialogService, ShellDialogService>();

        // ViewModels
        builder.Services.AddTransient<HomeViewModel>();
        builder.Services.AddTransient<ChatViewModel>();
        builder.Services.AddTransient<HistoryViewModel>();
        builder.Services.AddTransient<DetailViewModel>();
        builder.Services.AddTransient<SettingsViewModel>();
        builder.Services.AddTransient<TemplatesViewModel>();

        // Vistas
        builder.Services.AddTransient<HomePage>();
        builder.Services.AddTransient<ChatPage>();
        builder.Services.AddTransient<HistoryPage>();
        builder.Services.AddTransient<DetailPage>();
        builder.Services.AddTransient<SettingsPage>();
        builder.Services.AddTransient<TemplatesPage>();

#if DEBUG
```

Regenerar la documentación en PDF

```
pip install reportlab
python docs/informes/generar_informes.py     # informes de pruebas, seguridad y riesgos
python docs/informes/generar_documentos.py   # requisitos, arquitectura, clases, metodología, REST, SQLite, pantallas, manual t
python docs/manuales/generar_manual.py       # manual de usuario
```